Q1

VM - { hw1.1 }

```
Please describe the key applied fields with level of abstraction and
virtualization. Comparing the same and difference for these two approaches.
```

Based on the provided sources, the key applied fields for **abstraction** and **virtualization** are foundational to managing computer system complexity, compatibility, and resource flexibility. While both use well-defined interfaces to decouple system layers, they serve distinct purposes in how they present and manage technical details.

Key Applied Fields

1. Applied Fields of Abstraction

Abstraction is primarily used to **manage complexity** by dividing a system into hierarchical levels separated by well-defined interfaces.

- **Operating Systems (OS):** The OS abstracts complex hardware details into simpler software representations. For example, a physical hard disk (divided into tracks and sectors) is abstracted as a set of variable-sized **files** for application software.
- **Computer Architecture:** The **Instruction Set Architecture (ISA)** serves as the abstraction boundary between hardware and software. It allows hardware designers (like Intel) and software engineers (like Microsoft) to work independently as long as they adhere to the same ISA specification.
- **Application Programming Interfaces (APIs):** APIs provide a high-level abstraction of system services (like memory management or I/O) at the source code level, enabling software to be easily ported via recompilation.

2. Applied Fields of Virtualization

Virtualization is applied to **increase flexibility** and circumvent compatibility or hardware resource constraints.

- **Process Virtual Machines:** These support individual guest applications on different host platforms (e.g., running Windows/IA-32 apps on an Apple Macintosh).
- **System Virtual Machines:** These provide a complete system environment to support a full guest OS, allowing multiple operating systems (like Linux and Windows) to run simultaneously on the same hardware for **server consolidation** or **security isolation**.

- **High-Level Language VMs (HLL VMs):** Applied in frameworks like **Java** and **Microsoft .NET**, these provide full platform independence and secure network computing by executing abstract "virtual ISA" code.
- **Hardware Innovation (Codesigned VMs):** These use virtualization to implement entirely new, non-standard ISAs (like VLIW) while maintaining compatibility with standard software, often to improve power efficiency or performance.

Comparison: Abstraction vs. Virtualization

The sources distinguish these two approaches based on their relationship with system detail and interfaces.

Feature	Abstraction	Virtualization
Core Concept	Simplifies lower-level details into a higher-level representation.	Maps a virtual system's interface and resources onto a real system.
Detail Management	**Hides details** to simplify design (e.g., sectors become files).	**Does not necessarily hide details**; the virtual system often provides the same level of detail as the real one.
Detail Level	Operates across different levels of detail.	Typically provides a different interface at the **same level of abstraction**.
Primary Goal	Complexity management through simplification.	Flexibility, resource replication, and compatibility.

Similarities

- **Interface Dependency:** Both rely on **well-defined interfaces** to decouple computer subsystems and allow them to interact independently.
- **Mathematical Model:** Both can be formally characterized by an **isomorphism**—a mapping function that links the state and operations of one system (guest/logical) to another (host/physical).

Differences

The fundamental difference lies in the **level of detail** provided. **Abstraction** is intended to simplify; for instance, a programmer interacting with a "file" does not need to know the physical addressing of a disk. In contrast, **virtualization** provides a mapping without simplification; a "virtual disk" might still provide track and sector addressing, exactly like a real disk, but those virtual sectors are mapped (via an isomorphism) to a physical host file. Thus, virtualization

focuses on providing a **different or replicated version** of a resource rather than a simplified one

Q2

```
請圖解敘述 Goldberg 對虛擬化之 Formal description.
```

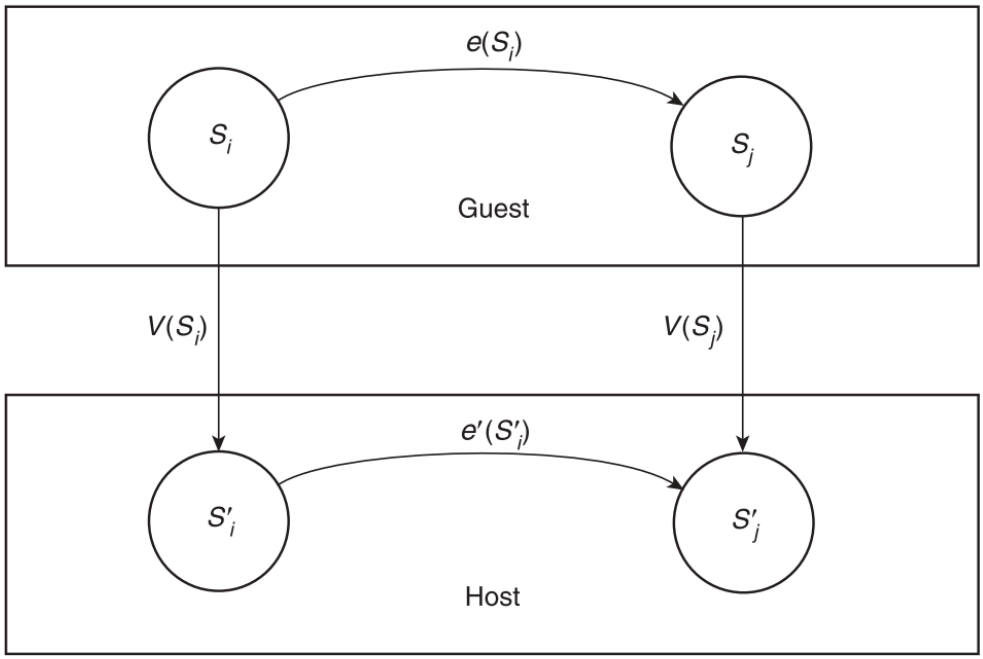

Figure 1.2 Virtualization. *Formally, virtualization is the construction of an isomorphism between a guest system and a host;* $e' \circ V(S_i) = V \circ e(S_i)$.

根據來源檔案，**Goldberg**（與 Popek 在 1974 年共同提出）對虛擬化（Virtualization）的正式描述是基於**同構（Isomorphism）**的概念。

這種正式描述透過建立一個對映關係，將虛擬的「客體（Guest）」系統對應到真實的「主體（Host）」系統。以下為其圖解敘述與詳細說明：

虛擬化同構圖解 (依據 Figure 1.2)

這是一個狀態轉換的對應模型，展示了客體系統的操作如何同步反映在主體系統中：

```
  客體系統 (Guest)                主體系統 (Host)
--------------------            --------------------
狀態 S_i -------- e(S_i) --------> 狀態 S_j
   |                                |
   |                                |
   | V(S_i)                         | V(S_j)
   |                                |
   v                                v
狀態 S'_i ------- e'(S'_i) ------> 狀態 S'_j
```

關鍵組件說明

1. **狀態對映函數** V：
   - 將客體系統的狀態（Si）映射到主體系統的對應狀態（Si′）。
   - 這包括暫存器、記憶體或檔案等資源的映射。
2. **客體操作** e：
   - 在客體系統中執行的一個操作序列（例如一條指令），使客體狀態從 Si 轉變為 Sj。
3. **主體操作** e′：
   - 在主體系統中執行的對應操作序列（例如模擬指令或系統呼叫），使主體狀態從 Si′ 轉變為 Sj′。

正式定義的邏輯要求

根據來源，一個成功的虛擬化必須滿足以下數學關係： $e' \circ V(Si) = V \circ e(Si)$

這表示：

- **路徑 A**：先將客體狀態 Si 透過 V 映射到主體，再執行主體操作 e′。
- **路徑 B**：先在客體執行操作 e 改變狀態，再透過 V 映射到主體。
- **結論**：無論走哪條路徑，最終在主體系統上產生的狀態結果必須是**等效**的。

虛擬化與抽象化（Abstraction）的區別

雖然同構模型也可用於描述抽象化，但 Goldberg 強調兩者不同：

- **抽象化**通常會隱藏底層細節（例如將硬碟磁區抽象化為檔案）。
- **虛擬化**不一定會隱藏細節。虛擬系統提供的細節層次通常與底層真實系統完全相同（例如虛擬磁碟仍提供磁軌/磁區的定址介面）。

Q3

VM - { hw1.3 }

```
請圖示 Computer System Architecture 有那些 implementation layer
提供那些 interface 被常應用於 Virtual Machine?
```

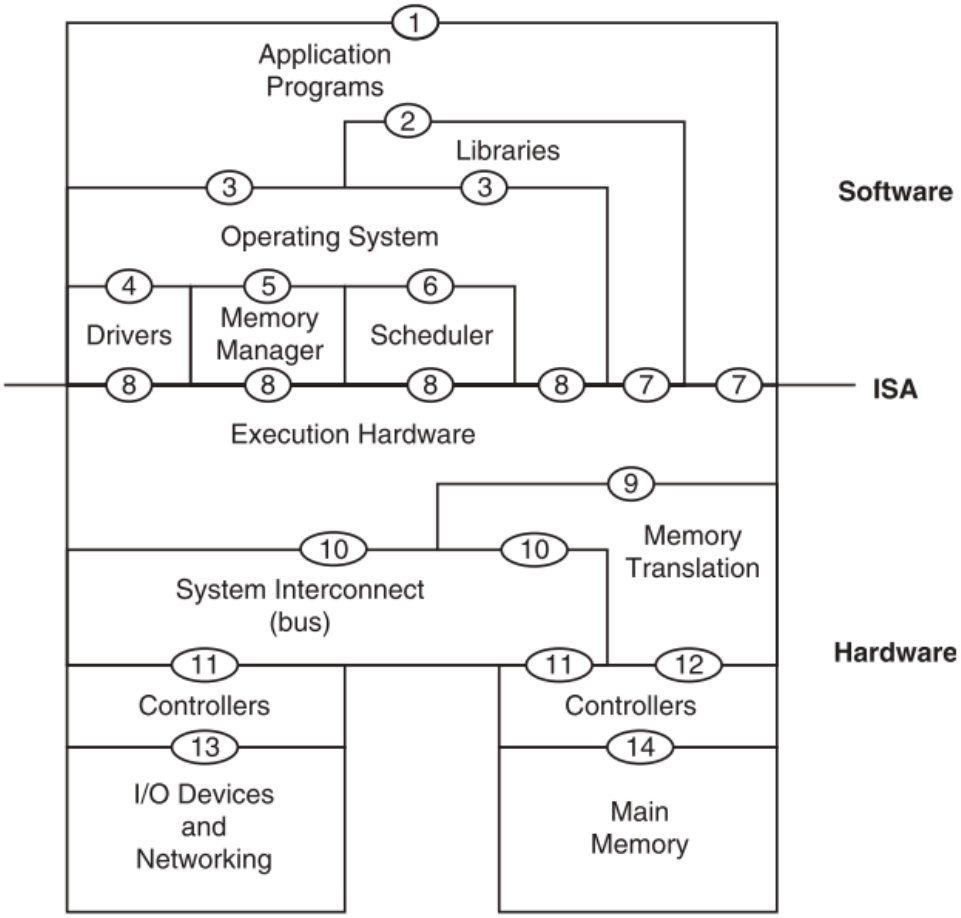

Figure 1.4 Computer System Architectures. *Implementation layers communicate vertically via the shown interfaces. This view of architecture is styled after one given by Glenford Myers (1982).*

根據來源檔案（特別是 **Figure 1.4**），電腦系統架構由多個**實作層（Implementation Layers）**組成，層與層之間透過**介面（Interfaces）**進行垂直溝通。虛擬化技術正是應用在這些介面上，以提供靈活性與相容性。

以下為電腦系統架構的層次與介面圖示敘述：

電腦系統架構實作層與介面圖示

```
實作層 (Implementation Layers)            應用之介面 (Interfaces)
-----------------------------            ----------------------
[    應用程式 (Applications)    ]
                                  <----  (2) API (應用程式介面)
[      函式庫 (Libraries)      ]
                                  <----  (3) 系統呼叫 (System Calls)
[    作業系統 (Operating Sys)  ]
================================= <----  (ABI) 應用程式二進位介面 [3+7]
                                  <----  (7) 使用者 ISA (User ISA)
[    硬體執行 (Exec Hardware)   ]
================================= <----  (ISA) 指令集架構 [7+8]
```

```
                                        <----  (8) 系統 ISA (System ISA)
    [ 硬體管理層 (Memory/IO/Bus)   ]
```

關鍵實作層 (Implementation Layers)

根據來源，主要層次包括：

- **軟體層**：應用程式、標準函式庫、以及負責管理硬體資源的作業系統。
- **硬體層**：執行硬體（Execution Hardware）、記憶體轉換（Memory Translation）、系統互連匯流排（Bus）以及 I/O 裝置控制器。

常見於虛擬機器的介面 (Interfaces)

虛擬化技術主要應用於以下三個關鍵介面，用以區隔軟體與底層機器：

1. **ISA (指令集架構)**：
   - **組成**：包含**使用者 ISA (介面 7)** 和**系統 ISA (介面 8)**。
   - **虛擬機應用**：**系統虛擬機器 (System VMs)** 應用於此層級。虛擬化軟體（VMM）位於硬體之上，模擬完整的 ISA，使多個客體作業系統能同時執行。
2. **ABI (應用程式二進位介面)**：
   - **組成**：由**使用者 ISA (介面 7)** 與**系統呼叫介面 (介面 3)** 組成。
   - **虛擬機應用**：**程序虛擬機器 (Process VMs)** 應用於此層級。虛擬化軟體（Runtime）位於作業系統/硬體組合之上，為個別應用程式提供虛擬執行環境。
3. **API (應用程式開發介面)**：
   - **組成**：通常指原始碼層級的標準函式庫呼叫（如 **介面 2**）與使用者指令集。
   - **虛擬機應用**：**高階語言虛擬機器 (HLL VMs)**（如 Java 或 .NET）應用於此。它們透過定義「虛擬 ISA (V-ISA)」與標準 API 函式庫，實現跨平台硬體與作業系統的獨立性。

總結來說，不同的虛擬機類型根據其目標（是支援單一程序還是完整系統）選擇在不同的介面層級進行攔截與模擬，藉此達成資源共用、安全性隔離或跨平台相容等目的。

Q4

VM - { hw1.4 }

```
對 Process VM, High Level Language VM, Hosted System VM, Whole System VM, Co-designed VM 簡述其各別的範例與其應用.
```

根據來源檔案，以下為 Process VM、High-Level Language VM、Hosted System VM、Whole System VM 及 Co-designed VM 的範例與應用簡述：

1. 程序虛擬機器 (Process VM)

- **範例**：**Digital FX!32**（在 Alpha 平台上執行 Windows IA-32 應用程式）、**IA-32 EL**（在 Itanium 平台上支援 IA-32 程式）、**Aries**（支援 PA-RISC 程式於 Itanium 平台）。
- **應用**：主要提供跨平台的軟體相容性，允許原本為特定作業系統和指令集架構（ISA）編譯的二進制執行檔，在不同的主體平台上執行。此外，也應用於**同一架構的動態二進制最佳化**（如 HP Dynamo），在程式執行時即時提升效能。

2. 高階語言虛擬機器 (High-Level Language VM, HLL VM)

- **範例**：**Java 虛擬機器 (JVM)**、**Microsoft .NET 公用語言基礎架構 (CLI)**、早期的 **Pascal P-code VM**。
- **應用**：旨在實現完全的**平台獨立性**與網路計算安全。開發者只需將程式編譯成虛擬 ISA（如 Java bytecode），即可在任何安裝了該 HLL VM 的硬體/作業系統上執行，廣泛應用於網際網路應用程式、行動裝置及企業軟體開發。

3. 主體裝載系統虛擬機器 (Hosted System VM)

- **範例**：**VMware Workstation/GSX Server**、**Virtual PC**。
- **應用**：這種虛擬機安裝在現有的主體作業系統（如 Windows 或 Linux）之上，應用於**多作業系統環境測試**、軟體開發以及伺服器整合。其優勢在於能直接利用主體作業系統提供的裝置驅動程式，簡化硬體支援的複雜度。

4. 全系統虛擬機器 (Whole System VM)

- **範例**：**Virtual PC** 於 Macintosh 平台（在 PowerPC 硬體上執行完整的 Windows 系統及其應用程式）。
- **應用**：當主體與客體系統的作業系統與指令集完全不同時，虛擬化整個軟體堆疊（包含作業系統和所有應用程式）。這常用於在非相容硬體上模擬完整電腦環境，解決**遺留軟體支援**或跨平台辦公需求。

5. 共同設計虛擬機器 (Co-designed VM)

- **範例**：**Transmeta Crusoe**、**IBM AS/400 (iSeries)**、IBM Daisy 研究系統。
- **應用**：硬體與虛擬軟體（VMM）同步開發，用於**硬體架構創新**。例如，Transmeta 使用 VLIW 核心配合軟體模擬 IA-32 指令以降低功耗；IBM 則藉此維持軟體介面穩定，同時在底層自由更換硬體架構（如從專屬 CISC 轉向 PowerPC）。其核心目標通常是**高效能、低功耗或設計簡化**。

Q5

VM - { hw1.5 }

```
Interpretation 技術被分類為那幾種方式 ? 請比較列出各方式及其優缺點.
```

根據來源檔案，詮釋（Interpretation）技術主要被分類為以下幾種方式，其演進過程主要是為了減少解碼與跳躍指令帶來的效能開銷：

1. 解碼與發送詮釋 (Decode-and-Dispatch Interpretation)

這是最基本且直接的詮釋方式。它圍繞著一個中央循環（loop）結構，該循環抓取指令、解碼指令，然後根據指令類型「發送（dispatch）」到對應的詮釋常式（routine）。

- **優點**：
  - **記憶體需求低**：每種指令類型僅需一個詮釋常式。
  - **啟動速度快**：不需要對原始二進制碼進行預處理或轉換，幾乎沒有啟動延遲。
  - **可攜性好**：若使用高階語言編寫，非常容易移植到不同平台。
- **缺點**：
  - **執行速度慢**：每次執行指令都必須重新解析（parsing）。
  - **快取壓力大**：源代碼被視為資料讀取，對資料快取壓力大。
  - **分支預測效率低**：中央循環中包含大量分支指令（如 `switch` 語句），導致效能損失。

2. 間接執行緒詮釋 (Indirect Threaded Interpretation)

為了減少中央發送循環的負擔，此技術將解碼與發送的程式碼附加在每個詮釋常式的結尾。它透過一個中央發送表（dispatch table）來間接跳轉到下一個常式。

- **優點**：
  - **效能略高於基本方式**：消除了中央循環，並減少了部分分支指令。
  - **維護靈活性**：詮釋常式可以獨立修改或重新定位，因為是透過表格間接存取。
  - **可攜性佳**：與基本詮釋方式同樣具備良好的移植性。
- **缺點**：
  - **記憶體需求略增**：因為發送代碼被複製到了每個常式中。
  - **仍有查表開銷**：每次跳轉仍需存取發送表，這涉及一次記憶體存取與間接分支指令。

3. 預先解碼 (Predecoding)

這通常是與執行緒詮釋搭配使用的預處理步驟。在執行前，先將源指令解析成一種更容易處理的中間形式（Intermediate Form），例如將操作碼與暫存器欄位重新排列以利直接存取。

- **優點**：
  - **減少重複解析**：重複執行的指令不需每次都重新解析欄位。
  - **簡化常式邏輯**：詮釋常式操作中間形式時會比操作原始 CISC 指令更簡單。
- **缺點**：
  - **啟動速度慢**：需要先對原始碼進行解讀以發現控制流，轉換過程耗時。
  - **記憶體需求高**：需要空間儲存中間形式的映像檔。

4. 直接執行緒詮釋 (Direct Threaded Interpretation)

這是預先解碼的進階應用。它不使用發送表，而是將中間形式中的操作碼直接替換為對應詮釋常式的「實際硬體記憶體位址」。

- **優點**：
  - **穩定狀態效能最佳**：完全消除了發送表的查表開銷（Memory lookup），直接跳轉到常式位址。
- **缺點**：
  - **可攜性中等/較差**：中間形式變得依賴於詮釋常式在記憶體中的具體位置。若移植到新機器，中間碼必須重新生成。
  - **記憶體需求高**：同樣受預先解碼產生的空間開銷影響。

詮釋技術比較表總結 (依據來源 Section 2.10)

技術	啟動效能	穩定狀態執行效能	記憶體需求	程式碼可攜性
解碼與發送	極快	慢	低	優
間接執行緒	極快	慢（略優於前者）	低	優
預先解碼+直接執行緒	慢	中等（詮釋中最快）	高	中等

Q6

VM - { hw1.6 }

```
簡述 Dynamic Binary Translation 的架構與執行流程, 比起 interpretation 兩者在效率上的差距來自何處?
```

動態二進位翻譯（Dynamic Binary Translation, DBT）是一種高效的模擬技術，透過將源指令集架構（Source ISA）的程式碼區塊即時轉換為目標指令集架構（Target ISA）來執行。

以下簡述其架構、執行流程，以及與詮釋（Interpretation）技術的效率差異來源：

1. 動態二進位翻譯的架構與組件

根據來源，一個典型的動態翻譯系統包含以下關鍵組件：

- **模擬管理器（Emulation Manager, EM）**：負責高層級的控制流，決定何時進行詮釋、翻譯或直接執行已快取的代碼。

- **詮釋器（Interpreter）**：在程式啟動初期執行原始二進位碼，並負責「代碼發現（Code Discovery）」，找出程式的執行路徑。
- **翻譯器（Translator）**：將源指令區塊（如動態基本區塊，Dynamic Basic Block）轉換為目標架構的等效指令。
- **代碼快取（Code Cache）**：存放已翻譯好的目標代碼區塊，以便後續重複執行。
- **對應表（Map Table）**：記錄源程式計數器（SPC）與目標程式計數器（TPC）的關聯，用於快速檢索快取中的代碼。
- **剖析資料庫（Profile Database）**：收集執行統計數據（如區塊執行頻率），用以識別「熱點（Hotspots）」並觸發更高層級的最佳化。

2. 執行流程 (Execution Flow)

執行流程通常採取「階段式模擬（Staged Emulation）」策略：

1. **初始詮釋**：程式啟動時，模擬管理器先由詮釋器逐條執行源指令，並同時進行剖析（Profiling）。
2. **熱點偵測**：當某個代碼區塊的執行次數達到預設門檻（如執行超過 50 次），模擬管理器會判定其為「熱點」。
3. **轉換與快取**：翻譯器將該熱點區塊翻譯成目標架構碼，存入代碼快取，並在對應表中建立索引。
4. **直接執行**：後續若程式流向已翻譯的位址，模擬管理器會查表命中（Hit），直接跳轉到代碼快取中，讓硬體原生執行目標指令。
5. **連結與最佳化**：翻譯區塊間可以透過「鏈結（Chaining）」技術直接串聯，減少頻繁回到模擬管理器的開銷。
6. 與詮釋技術的效率差距來源

翻譯技術在穩態執行（Steady-state）效能上大幅領先詮釋技術，主因如下：

- **執行媒介不同**：
  - **詮釋器**：每一條源指令的擷取、解碼與模擬都在**軟體層級**重複進行，每條源指令可能需耗費數十條原生指令來完成。
  - **二進位翻譯**：將轉換成本攤銷（Amortized），翻譯後的代碼區塊直接在**硬體上以原生速度執行**。
- **記憶體架構壓力**：
  - 詮釋器將源代碼視為「資料」讀取，會對**資料快取（Data Cache）**造成極大壓力。
  - 翻譯後的代碼則像一般程式一樣透過**指令快取（Instruction Cache）**抓取，更符合處理器硬體設計的效能特性。
- **最佳化機會**：
  - 詮釋器通常只能觀察單一指令。

    - 翻譯技術可以針對整個區塊進行最佳化，例如**消除冗餘指令（Common Subexpression Elimination）、指令重排（Code Reordering）**暫存器映射（State Mapping）**，進一步提升效率。
- **效能權衡**：詮釋技術雖然啟動快（零負擔），但執行慢；翻譯技術雖然啟動開銷高（需預先翻譯），但一旦完成後，重複執行的速度極快。

Q7

VM - { hw1.7 }

```
請簡述 shade simulation tool 與 emulation 的關係, 她使用那種技術提升性能?
```

Shade 是一個專為高效能模擬而開發的工具，它與模擬（Emulation）的關係在於其內部包含一個複雜的模擬器，作為實現彈性且可擴展模擬的基礎。

Shade 與 Emulation 的關係

1. **模擬是核心組件**：Shade 在進行系統模擬（如研究記憶體階層或處理器行為）時，必須先透過模擬器執行基準程式。
2. **ISA 翻譯**：Shade 會將被模擬的指令集架構（Source ISA）翻譯成目標架構（Target ISA）的指令序列來執行。
3. **超越單純模擬**：雖然 Shade 包含模擬功能，但其目標是研究計算過程而非結果，因此會在翻譯後的代碼中嵌入函數呼叫，以收集追蹤資料（Trace data）。
4. **低優化需求**：與一般的虛擬機不同，Shade 通常只包含詮釋（Interpretation）與簡單翻譯技術，因為在模擬應用中，複雜的動態最佳化並非首要需求。

Shade 提升性能的技術

為了達成高效能模擬，Shade 使用了以下關鍵技術：

- **代碼快取（Trace Cache, TC）**：將翻譯後的目標指令塊存放在快取中，以便後續重複執行，避免重複翻譯的開銷。
- **暫存器永久映射**：將原始程式計數器（VPC）和原始記憶體映像（VMEM）的基址**永久映射到目標架構的硬體暫存器**中，以大幅提升存取速度。
- **翻譯鏈結（Chaining）**：翻譯塊之間可以直接串聯（Linked together），從而**繞過 TLB 查表**的過程，減少控制流轉換的開銷。
- **優化的翻譯旁看緩衝區（TLB）**：
    - 使用自定義的 TLB 結構來管理原始 PC 與目標 PC 的映射。
    - TLB 採用兩維陣列結構，並將**每一列與快取行邊界對齊**，確保在發生初次查表不中（Miss）時，剩餘的條目能同步載入快取以利後續搜索。
    - TLB 採用最近使用（MRU）策略，將最近命中的條目移至首位，加快搜索速度。

- **簡單的快取管理**：當代碼快取（TC）填滿時，直接將其清除（Flush）並重新開始，這種做法簡單且能有效清除因地圖表更新而產生的「孤兒（Orphan）」翻譯塊。

Q8

VM - { hw1.8 }

```
決定 compatibility 有那些條件? 請用 isomorphism 圖示 說明 Process VM 如何進行可達到相容性?
```

1. isomorphism 圖示

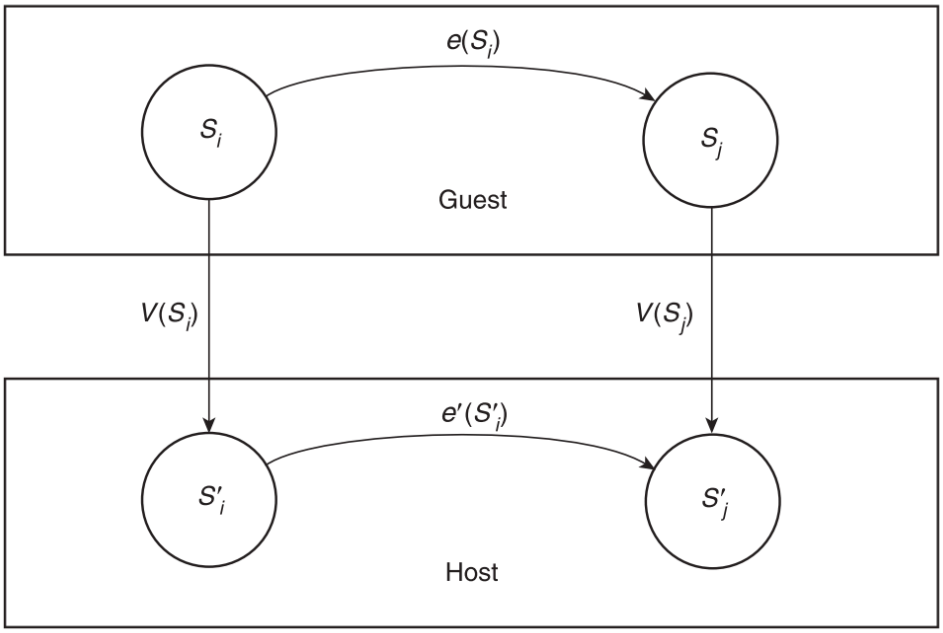

Figure 1.2 Virtualization. *Formally, virtualization is the construction of an isomorphism between a guest system and a host;* $e' \circ V(S_i) = V \circ e(S_i)$.

根據來源檔案，**相容性（Compatibility）**指的是客體（Guest）系統在主體（Host）平台上執行的行為，與在其原生（Native）平台上執行時的行為之準確度比較。決定相容性的條件與其實現方式如下：

2. 決定相容性的主要條件

相容性的定義排除單純的效能差異，專注於**功能的正確性**。根據來源，決定條件分為兩大類層級，以及兩個具體的充分條件：

- **相容性層級**：**
  - **本質相容性（Intrinsic Compatibility）：** 要求對**所有**客體軟體與所有可能的輸入數據都能 100% 準確模擬。這也稱為完全透明，通常是硬體設計者的標準。
  - **外在相容性（Extrinsic Compatibility）：** 僅針對客體軟體的**子集**（例如由特定編譯器產生的程式，或資源需求在特定限度內的程式）保證相容。
- **充分相容條件（Sufficient Conditions）**：**

1. **控制移交點的狀態等效：** 在控制權從模擬的使用者指令轉移到作業系統（OS）時（例如發生 Trap 或 OS 呼叫），客體狀態必須在給定的狀態對映下與主體狀態等效。
2. **回復執行點的狀態等效：** 在控制權從 OS 移回使用者指令時，客體狀態（包括使用者管理與 OS 管理的狀態）必須與主體狀態等效，且模擬產生的系統行為必須與原生平台一致。

重要洞察： 狀態等效性不需要在每一條指令的粒度下維持，只需要在 **OS 控制轉移點**維持即可，這為虛擬機的代碼最佳化提供了巨大的彈性。

---

2. Process VM 實現相容性的同構圖解 (依據 Figure 3.4)

程序虛擬機器（Process VM）透過在 **ABI（應用程式二進位介面）**層級攔截操作來實現相容性**。**以下為基於**同構（Isomorphism）**的執行流程圖解：

```
   客體系統（Guest）                原生 OS 轉移點
-------------------------------------------------------
狀態 S_i ----- e(S_i) ----> 狀態 S_j ----- e(S_j) ----> 狀態 S_k
   |        （使用者指令）       |         （OS 操作）        |
   |                            |                           |
   | V(S_i)                     | V(S_j)                    | V(S_k)
   | （狀態對映）                | （狀態等效）               | （狀態等效）
   v                            v                           v
狀態 S'_i ---- e'(S'_i) ---> 狀態 S'_j ---- e'(S'_j) ---> 狀態 S'_k
-------------------------------------------------------
   主體系統（Host）                模擬與主體 OS 轉移點
```

圖解流程說明

1. **狀態對映** V：
   - 將客體的暫存器、記憶體與檔案映射到主體資源中。例如，客體暫存器可能映射到主體的記憶體區塊中。
2. **使用者指令模擬** e′(Si′)：
   - 當客體執行使用者層級指令（e）使狀態從 Si 變為 Sj 時，Process VM 的**模擬引擎（Emulation Engine）**會執行主體指令序列（e′），確保在進入 OS 呼叫點時，主體狀態 Sj′ 與客體狀態 Sj 是等效的。
3. **OS 呼叫模擬** e′(Sj′)：
   - 當客體嘗試進行系統呼叫（e）時，**OS 呼叫模擬器（OS Call Emulator）**會攔截此請求，將其轉換為一或多個主體 OS 的系統呼叫（e′）。
   - 模擬器必須確保當控制權回到使用者指令時（狀態 Sk），所有對記憶體或檔案系統的修改都與原生平台產生的結果一致。

結論： Process VM 藉由維持這種在關鍵介面點的狀態對映與操作等效性，讓原本為不同 OS 或 ISA 編譯的程式能夠在非原生環境下正確執行。

Q9

VM - { hw1.9 }

```
Process VM 使用 State Mapping 技術, 需考慮那些問題, 請列出解法
```

在程序虛擬機器（Process VM）中，**狀態對映（State Mapping）** 是將客體（Guest）系統的資源（如暫存器、記憶體）對應到主體（Host）資源的核心技術。實作時主要需考慮暫存器與記憶體兩大層面的問題，以下列出其挑戰與解法：

1. 暫存器狀態對映 (Register Mapping)

暫存器位於儲存階層的最頂端，對效能至關重要，主要面臨資源數量不匹配的問題：

- **問題一：客體暫存器數量多於主體。**
  - **解法**：必須將部分客體暫存器映射到主體記憶體中的**暫存器內容塊（Register Context Block）**。編譯器或翻譯器需負責在執行時將這些暫存器內容換入或換出主體實際的暫存器。
- **問題二：客體與主體暫存器數量相等（或接近）。**
  - **解法**：虛擬機軟體（Runtime）本身也需要暫存器來執行模擬邏輯或進行最佳化。此時通常維持一個內容塊，在進入模擬模式時載入暫存器，退出到 Runtime 時再存回內容塊。此外，必須預留幾個主體暫存器作為指針（如指向內容塊或記憶體映像的基底位址）。
- **問題三：特殊暫存器（如狀態碼 Condition Codes）的不匹配。**
  - **解法**：若主體缺乏對等的狀態碼，必須透過軟體模擬（這非常耗時），或是採取**延遲評估（Lazy Evaluation）**，僅在後續指令確實需要該狀態碼時才進行計算。

2. 記憶體位址空間對映 (Memory Address Space Mapping)

由於客體程式與虛擬機 Runtime 共享同一個主體位址空間，對映技術決定了相容性與效能：

- **問題一：客體與主體位址空間大小不同（例如 64位元主體模擬 32位元客體）。**
  - **解法 A（高效能）：直接對映（Direct Mapping）**。若主體空間足夠大，可將客體位址直接映射到主體中相同的位址（零偏移），這樣載入/儲存指令不需額外轉換。
  - **解法 B（靈活）：軟體轉換表（Software Translation Tables）**。類似分頁表，將客體位址分成區塊映射到主體的非連續區域，雖然功能正確但速度較慢。
- **問題二：分頁大小不一致。**
  - **解法**：若客體分頁小於主體，多個客體分頁可能位於同一個主體分頁內，導致權限管理衝突。此時需採取**保守權限策略**（給予該主體分頁最嚴格的權限），並透過訊號處理常

式（Signal Handlers）來處理額外觸發的例外。

3. 安全與 Runtime 保護問題

由於客體程式具有任意讀寫記憶體的能力，必須防止其破壞虛擬機自身的內容：

- **問題：客體程式存取虛擬機 Runtime 的記憶體區域。**
  - **解法 A：軟體權限檢查**。在每條翻譯後的載入/儲存指令前插入檢查程式碼，確保位址在合法範圍，但這會增加約 10% 或更多的效能開銷。
  - **解法 B：硬體分頁保護**。在切換到模擬模式前，利用系統呼叫（如 `mprotect()`）將 Runtime 區域設為不可存取，將程式碼快取（Code Cache）設為唯執行，回到 Runtime 模式時再切換回讀寫權限。

4. 例外與精確狀態回復 (Precise State Recovery)

當發生 Trap 或中斷時，主體必須能呈現與客體原生平台一致的狀態：

- **問題：翻譯後的程式碼順序可能被更動，導致 Trap 發生時暫存器數值不正確。**
  - **解法**：在翻譯塊（Translation Block）的入口或關鍵點設置**檢查點（Checkpoints）**。透過維護 **RMAP（暫存器對映表）** 與側表（Side Tables），在發生例外時，Runtime 可以將狀態回溯（Back up）到最近的精確狀態點，再透過詮釋器（Interpreter）逐條執行原始碼以重現正確狀態。

Q10

VM - { hw1.10 }

```
Exception 與 interrupt 如何被 emulation? 如何精確確定 guest state?

若使用 binary translation 如何定位出 program counter? 對Process VM若使用不同的OS,
如何實現 windows callback function ?
```

根據來源檔案，關於虛擬機如何處理例外（Exceptions）、中斷（Interrupts）、精確狀態恢復、PC 定位以及 Windows 回呼函式（Callback）的實作方式說明如下：

1. 例外與中斷的模擬 (Emulation)

在虛擬機中，例外與中斷的處理方式根據其性質（Trap 或 Interrupt）以及虛擬機類型而定：

- **例外（Traps，由指令觸發）的偵測**：
  - **詮釋偵測（Interpretive Detection）**：在詮釋執行指令時，軟體直接檢查條件（例如：檢查除法指令的除數是否為零）。

    - **硬體副產品偵測**：利用主體（Host）硬體觸發 Trap，由主體作業系統產生訊號（Signal）並遞交給虛擬機的 Runtime 處理。
- **中斷（Interrupts，外部事件）的處理**：
    - 中斷通常允許一定的回應延遲（Latency）。在詮釋模式下，Runtime 會等到目前的詮釋常式執行完畢後再處理中斷。
    - 在**二進位翻譯（Binary Translation）**解除鏈結（Unlink）**，強制控制權回到 Runtime，以確保在規定的回應時間內處理中斷。
- **ABI 可見性**：如果例外在客體（Guest）OS 中是可見的（例如：客體註冊了訊號處理常式），Runtime 會模擬客體 OS 的行為，將控制權轉移至客體的訊號處理程式中。

2. 精確客體狀態 (Precise Guest State) 的確定

「精確狀態」要求在發生例外時，該指令之前的指令皆已執行完成，之後的指令皆未執行。

- **詮釋執行**：較容易實現，因為指令是逐條執行的，Source PC 會同步更新。發生例外時，Source PC 會指向當前出錯的客體指令。
- **二進位翻譯（低最佳化層級）**：Runtime 雖然不即時更新 Source PC，但會透過**側表（Side Tables）**記錄翻譯後的程式碼與原始碼的對應關係。
- **二進位翻譯（高最佳化/指令重排層級）**：
    - 使用**檢查點（Checkpoints）**：在翻譯塊入口或關鍵點保存狀態。
    - **暫存器映射表（RMAP）與生命週期擴展**：透過 RMAP 追蹤原始暫存器值，並在最佳化時擴展這些暫存器的生命週期，確保在發生 Trap 時能回溯到最近的檢查點，再透過詮釋器重新執行以重現精確狀態。

3. 二進位翻譯中 Program Counter (PC) 的定位

由於翻譯後的程式碼在主體機器上執行時使用的是主體 PC（Target PC, TPC），而非客體的 Source PC (SPC)，因此在例外發生時定位 SPC 是個挑戰：

- **反向翻譯表（Reverse Translation Table）**：虛擬機會維護一個側表，記錄 `<Target PC, Source PC>` 的對映關係。
- **最佳化側表結構**：為了節省空間，側表通常只記錄**翻譯塊（Translation Block）的起始位址**與形成該區塊的資訊（如指令數量或分支行為）。當例外發生於區塊中間時，Runtime 會透過側表找到區塊起點，然後對原始碼進行**分析或詮釋執行**，直到定位出真正引發例外的 SPC。

4. Process VM 中 Windows Callback Function 的實現

當客體與主體作業系統不同時（例如在 Linux 上跑 Windows 程式），處理像 Windows Callback 這種事件驅動的機制非常複雜：

- **Windows 回呼機制**：Windows OS 會將事件放入訊息佇列，並在特定點呼叫使用者註冊的「回呼處理常式（Callback Handler）」。這涉及從內核（Kernel）切換回使用者模式執行特定

的發送常式（Dispatch Routine）。

- **虛擬機的實作策略：**
  1. **攔截進入點：**Runtime 必須在回呼處理程式啟動前獲得控制權。這通常透過修改（Patch）OS 的 **Dispatch Routine**，在開頭插入一個跳轉指令（Jump）到 Runtime 來達成。
  2. **狀態保護：**在進入處理程式前，Runtime 會保存其自身未被 OS 自動保存的狀態。
  3. **偵測返回：**Runtime 必須監控特定的系統呼叫，以偵測回呼何時執行完畢並準備返回核心。在返回之前，Runtime 會恢復先前保存的狀態，確保對虛擬環境的全面控制。